

Recurrent Neural Network

Natural Language Processing

Ayesha Enayet

Definition

- A recurrent neural network, or RNN, is a deep [neural network](#) trained on sequential or time series data to create a [machine learning](#) model can make sequential predictions or conclusions based on sequential inputs.
- <https://www.ibm.com/topics/recurrent-neural-networks#:~:text=A%20recurrent%20neural%20network%2C%20or,conclusions%20based%20on%20sequential%20inputs>.

Introduction to ANN:

- ANN (Artificial Neural Networks) are powerful models for solving classification tasks.
- They consist of interconnected layers of neurons (input, hidden, output).
- Commonly used for tasks where inputs and outputs have fixed lengths.

Limitations of ANN

- ANNs are designed for fixed-size inputs and outputs, making them unsuitable for variable-length sequences such as time series data, text, and speech.
- ANN lacks memory of past information, meaning it cannot retain or use information from previous inputs for decision-making.
- This makes it inefficient for tasks where context is important, like language processing, where past words influence the meaning of current words.

- ANNs cannot handle sequential data efficiently because they treat each input independently.
- This makes ANNs unsuitable for tasks that require understanding of patterns over time, such as predicting stock prices, analyzing video frames, or machine translation.

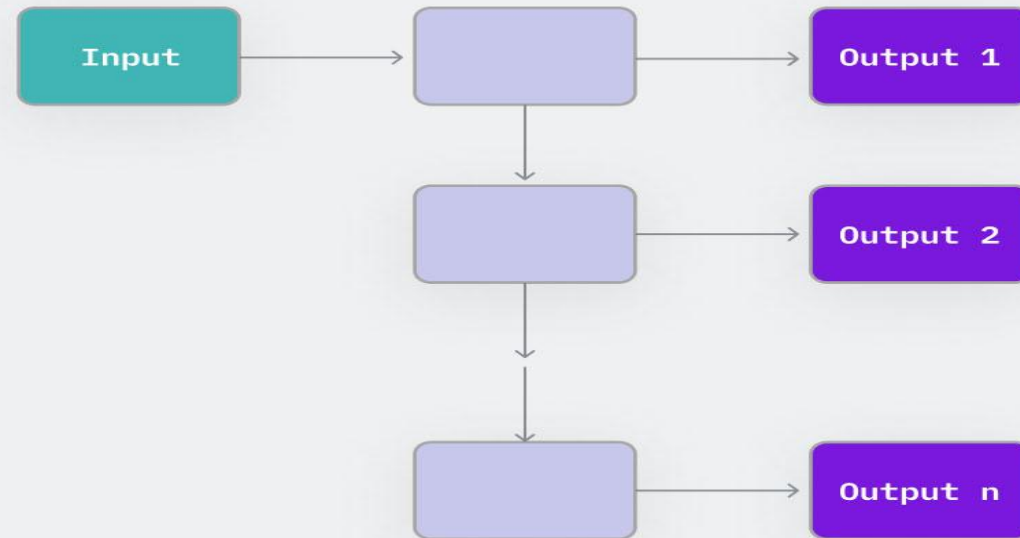
Types (One-to-One)

ANN's are more suitable for these type of tasks



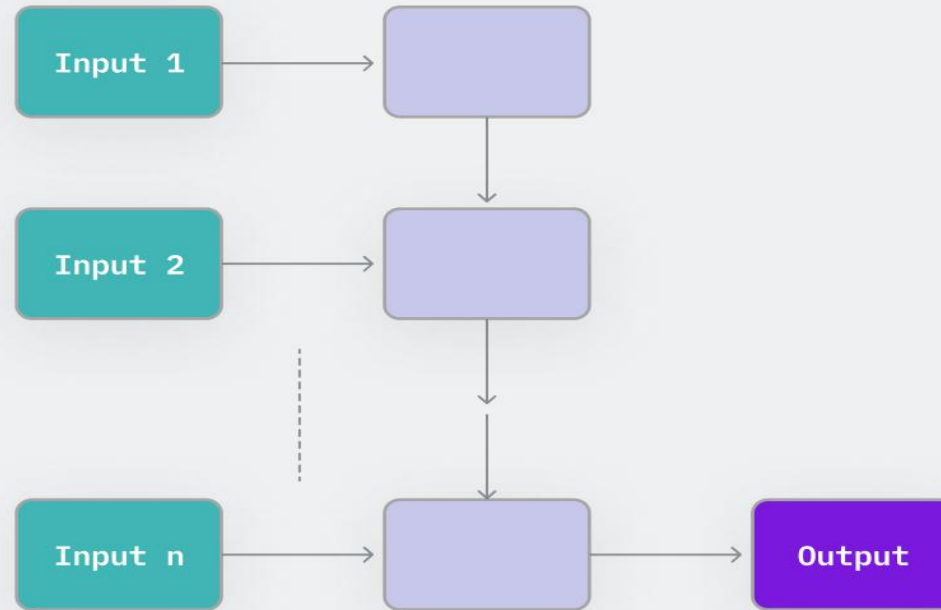
Types (One-to-Many)

We use RNNs or its variants for these type of tasks. e.g.
Image captioning



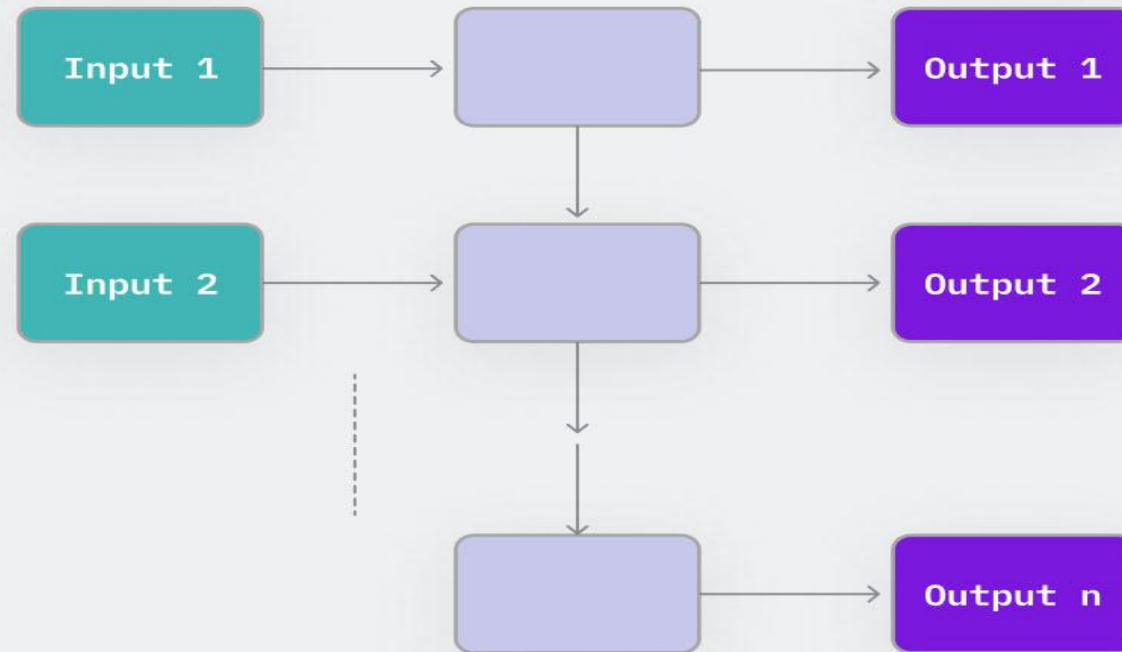
Types (Many-to-one)

We use RNNs or its variants for these type of tasks



Types (Many-to-many)

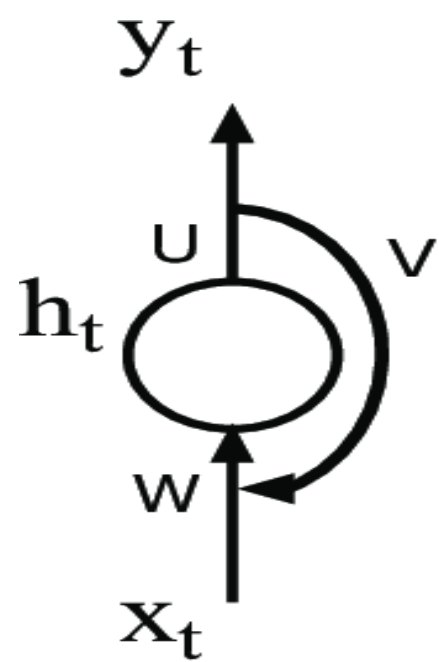
We use RNNs or its variants for these type of tasks



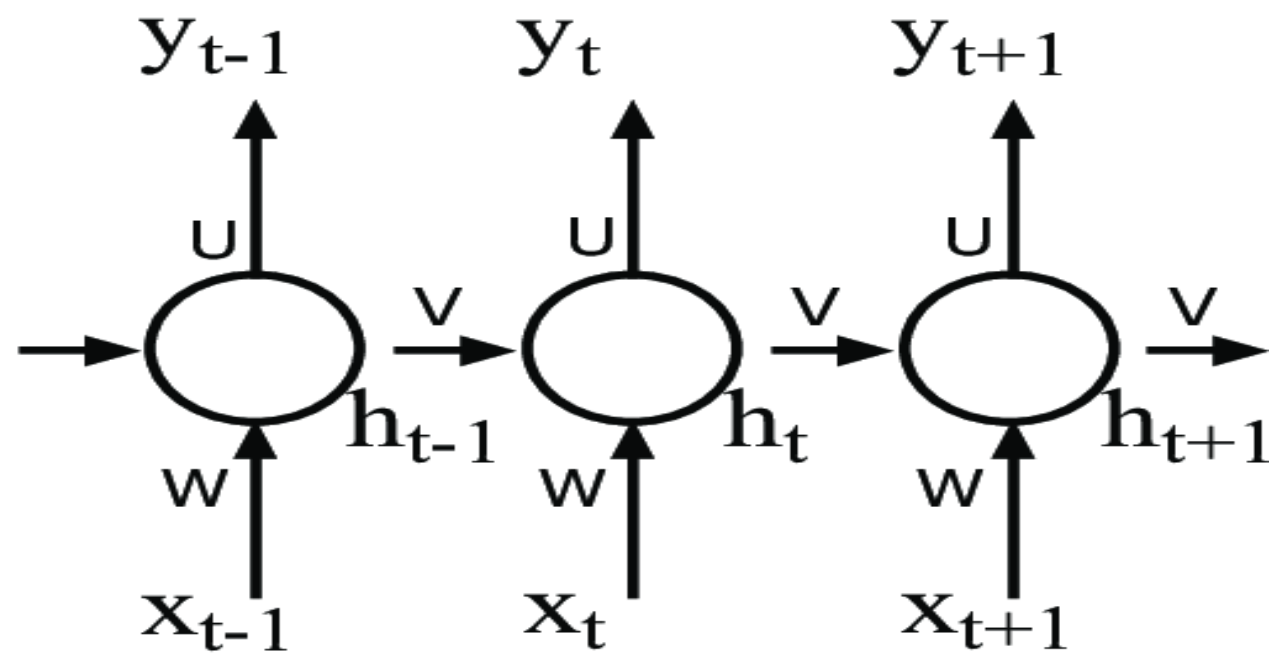
Visualization of RNN (Folded vs. Unfolded)

RNNs can be visualized in two ways:

- Folded: A simple loop representing the recurrent structure.
- Unfolded: Shows how the network progresses over multiple time steps.



Folded

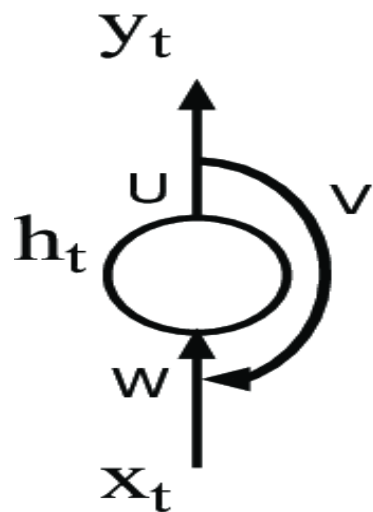


Unfolded

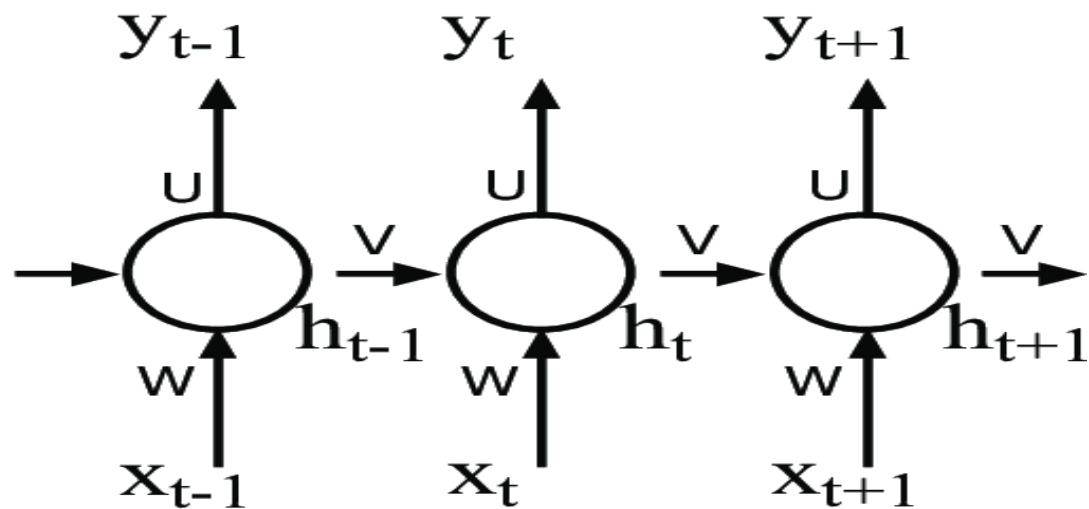
Unfolding

- For example, if the sequence we care about is a sentence of 3 words, the network would be unrolled into a 3-layer neural network, one layer for each word.

- $h_t = g(Vh_{t-1} + Wx_t)$
- $y_t = f(Uh_t)$
- $y_t = \text{Softmax}(Uh_t)$



Folded



Unfolded

Reading Quiz

- Write down the algorithm for Forward pass of RNN.

function FORWARDRNN($x, network$) **returns** output sequence y

$h_0 \leftarrow 0$

for $i \leftarrow 1$ **to** LENGTH(x) **do**

$h_i \leftarrow g(U h_{i-1} + W x_i)$

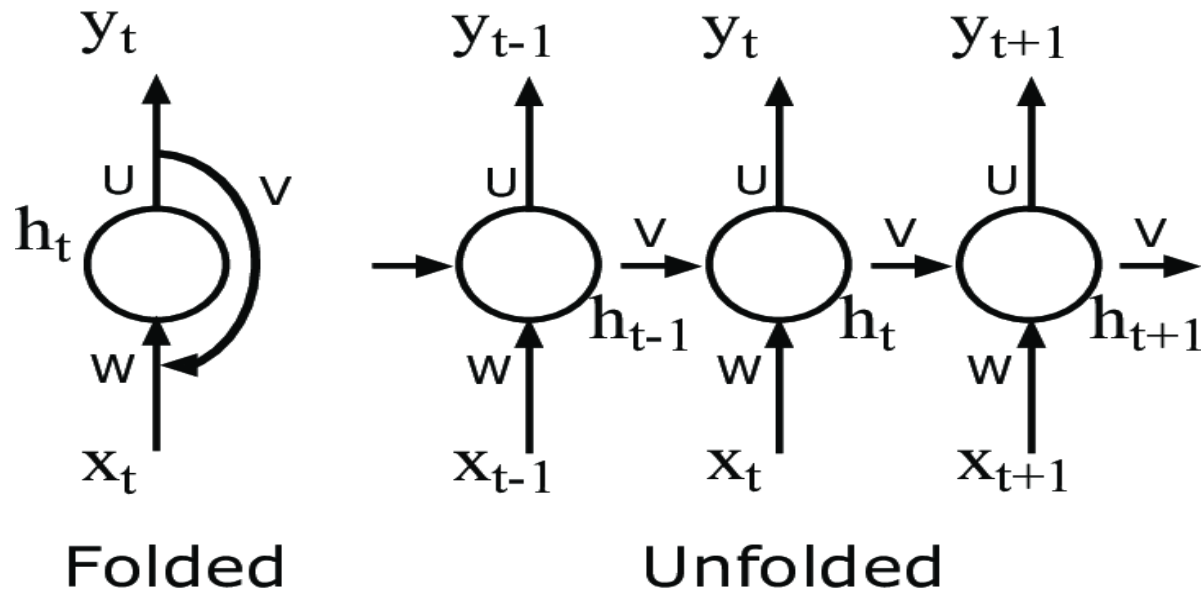
$y_i \leftarrow f(V h_i)$

return y

Figure 9.5 Forward inference in a simple recurrent network.

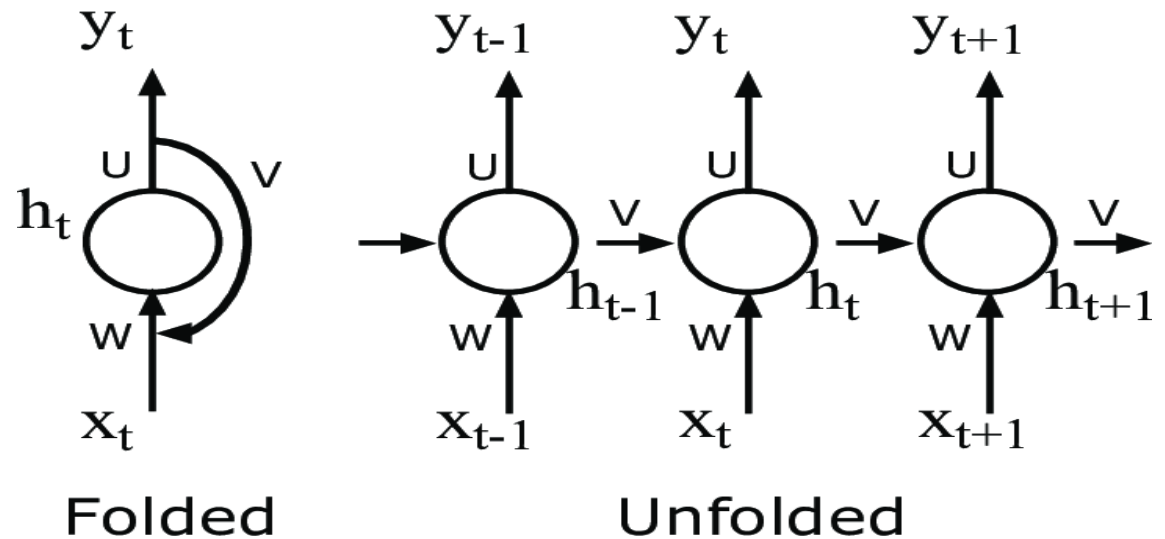
Input

- $x(t)$ is taken as the input to the network at time step t . For example, x_1 could be a one-hot vector corresponding to a word of a sentence.



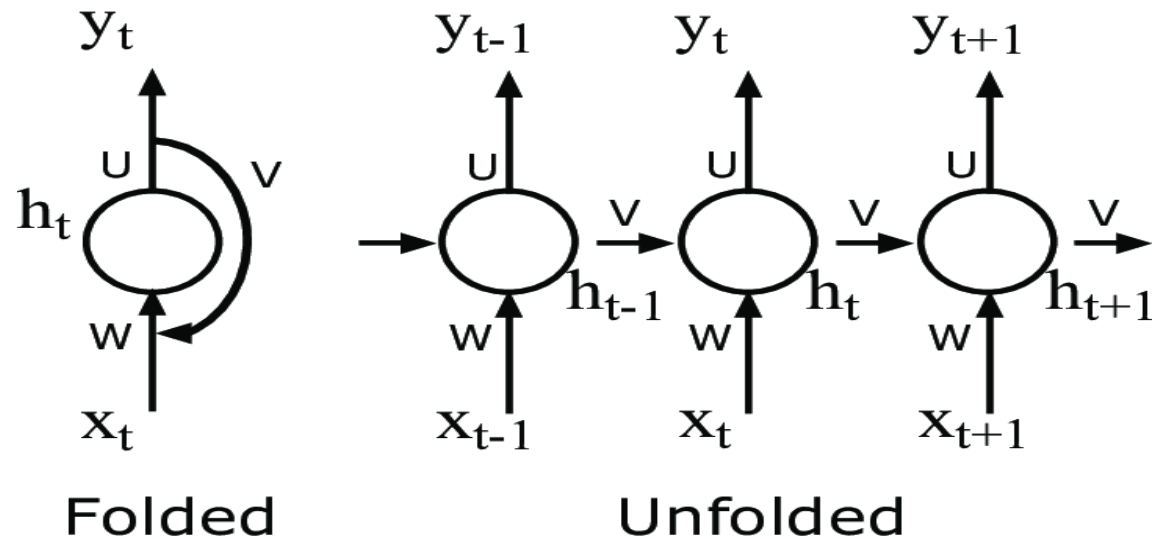
Hidden state:

- $h(t)$ represents a hidden state at time t and acts as “memory” of the network. $h(t)$ is calculated based on the current input and the previous time step’s hidden state: $h_t = g(Vh_{t-1} + Wx_t)$
The function g is taken to be a non-linear transformation such as *tanh*, *ReLU*.



Weights

- The RNN has input to hidden connections parameterized by a weight matrix W , hidden-to-hidden recurrent connections parameterized by a weight matrix V , and hidden-to-output connections parameterized by a weight matrix U and all these weights (U, V, W) are shared across time.



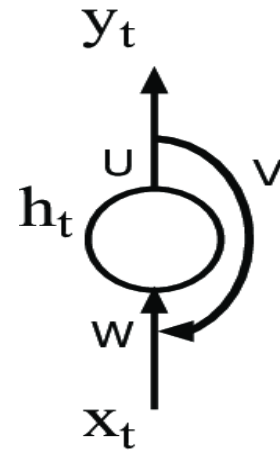
Output

- $y(t)$ illustrates the output of the network.

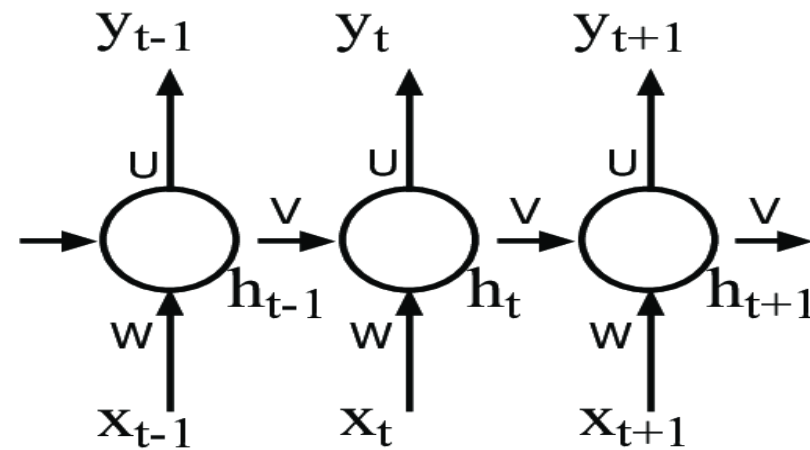
Dimensions

- Let's assume we pick a vocabulary size of 8000 and a hidden layer size of 100 then we have:

- $x_t = ?$
- $y_t = ?$
- $h_t = ?$
- $W = ?$
- $V = ?$
- $U = ?$



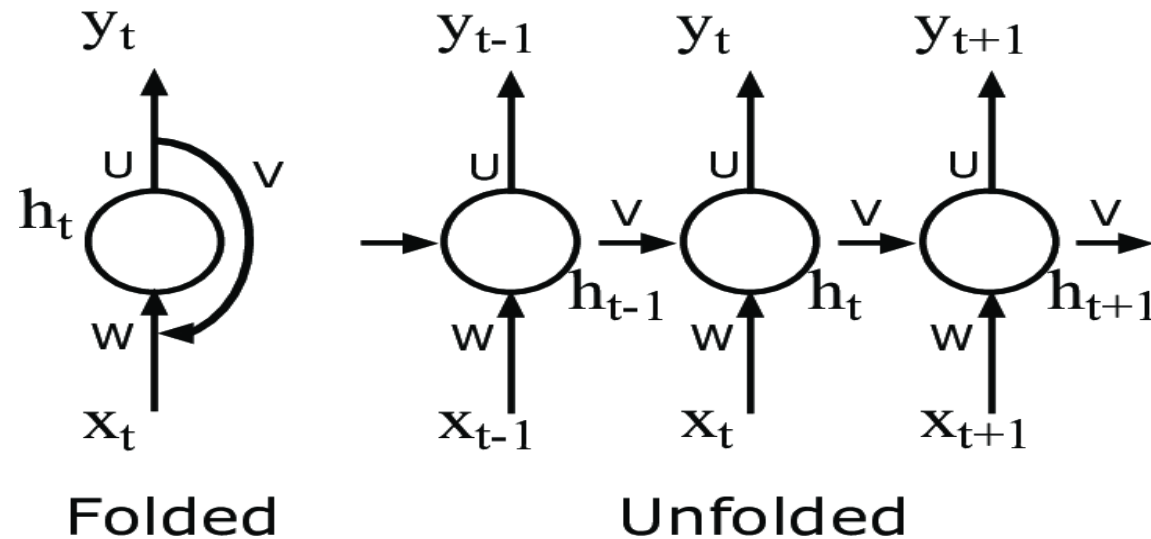
Folded



Unfolded

Dimensions

- Let's assume we pick a vocabulary size of 8000 and a hidden layer size of 100 then we have:
- $X_t = 8000 \times 1$
- $y_t = 42$ (POS tagging)
- $h_t = 100 \times 1$
- $W = 100 \times 8000$
- $V = 100 \times 100$
- $U = 42 \times 100$
- <https://towardsdatascience.com/recurrent-neural-networks-rnns-3f06d7653a85>



POS tagging example

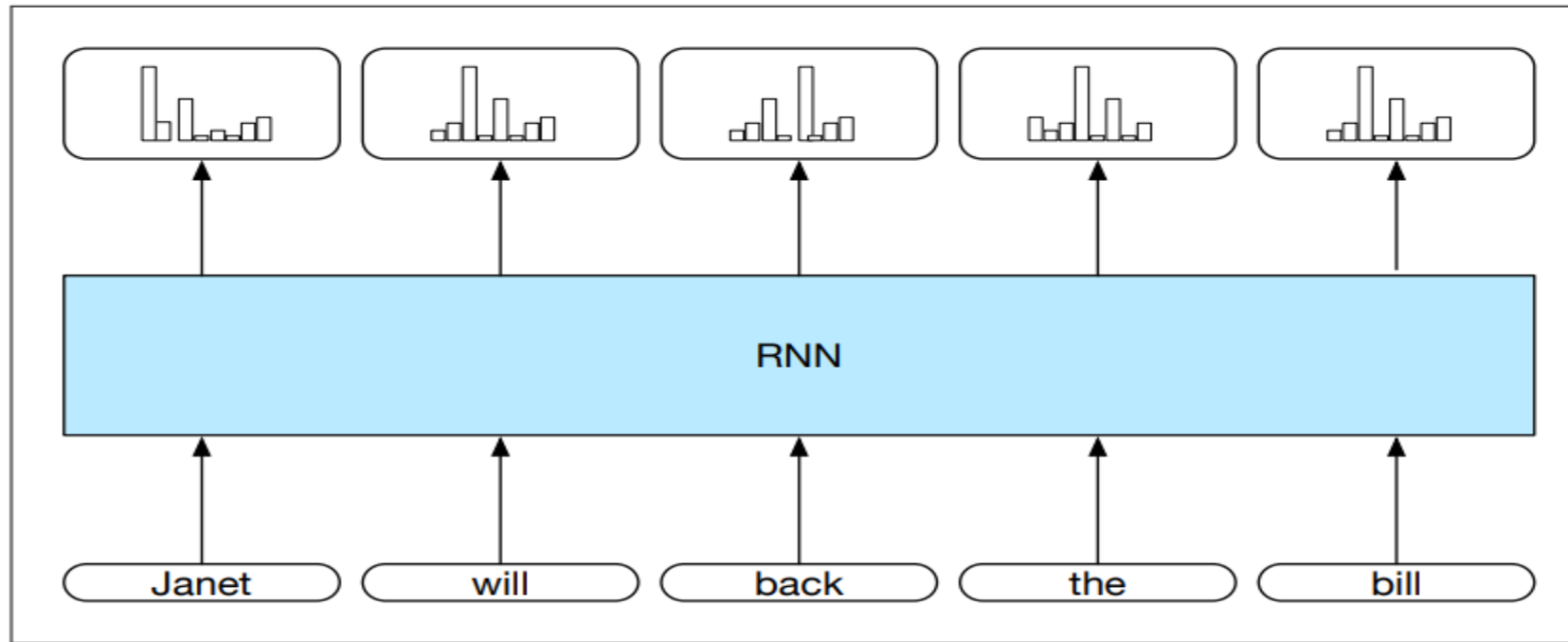


Figure 9.8 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

POS Tagging

- In a recurrent network-based approach to POS tagging, inputs are words and the outputs are tag probabilities generated by a softmax layer over the POS tag set.

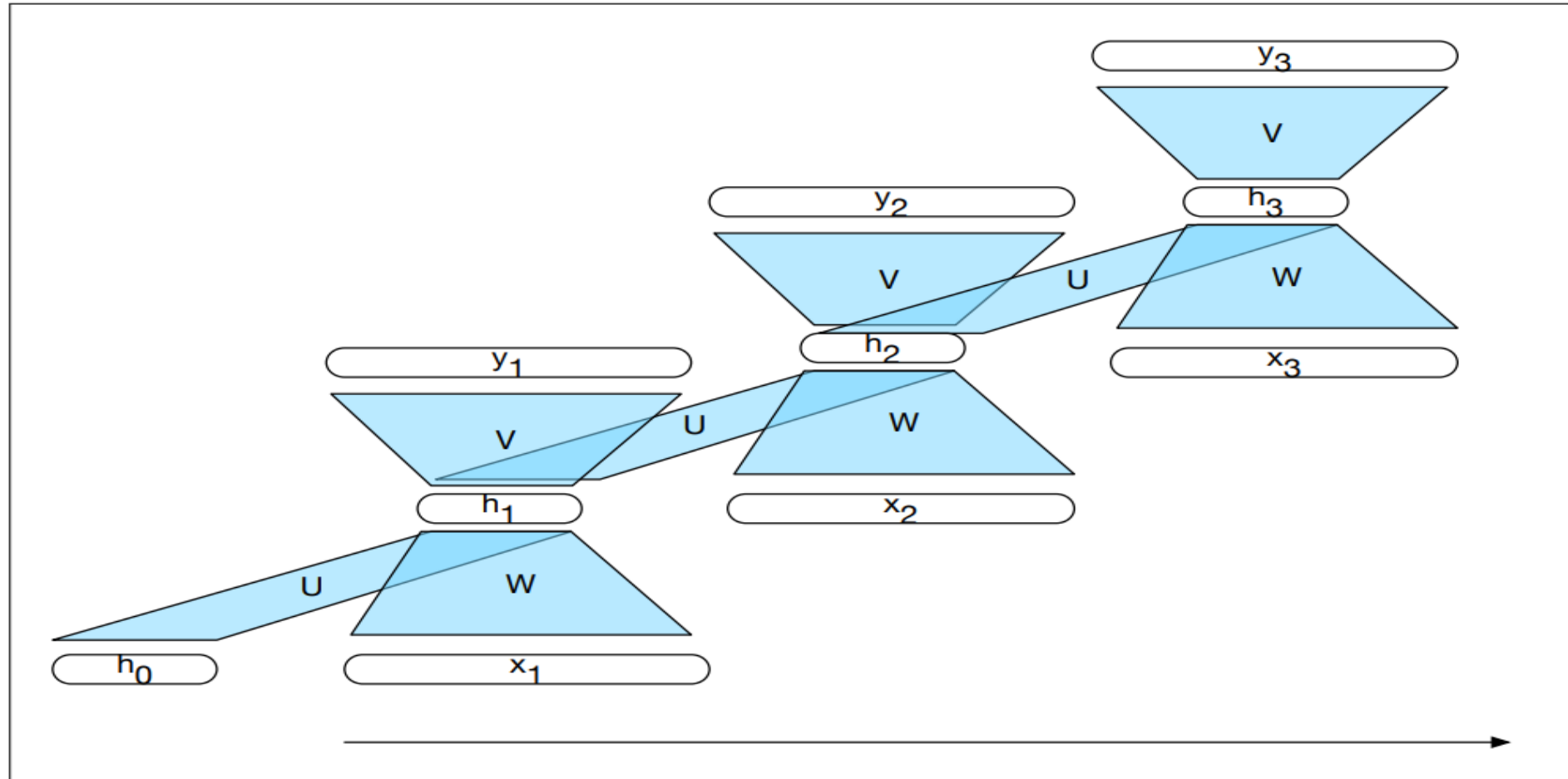


Figure 9.4 A simple recurrent neural network shown unrolled in time. Network layers are copied for each timestep, while the weights U , V and W are shared in common across all timesteps.

function FORWARDRNN($x, network$) **returns** output sequence y

$h_0 \leftarrow 0$

for $i \leftarrow 1$ **to** LENGTH(x) **do**

$h_i \leftarrow g(U h_{i-1} + W x_i)$

$y_i \leftarrow f(V h_i)$

return y

Figure 9.5 Forward inference in a simple recurrent network.

Basic Concept of RNN

- Recurrent Neural Networks (RNNs) are a type of neural network designed for sequential data.
- They have loops in the network that allow information to persist.

Structure of RNN

- In RNNs, the output of the current step depends on the input and the previous hidden state.
- This helps RNNs remember information from previous steps in the sequence.

How RNN Works (Step-by-Step)

- 1. Input is fed into the network at time step t .
- 2. The hidden state from the previous time step ($t-1$) is combined with the input.
- 3. The network processes the current input and updates the hidden state.
- 4. The process repeats for each time step.

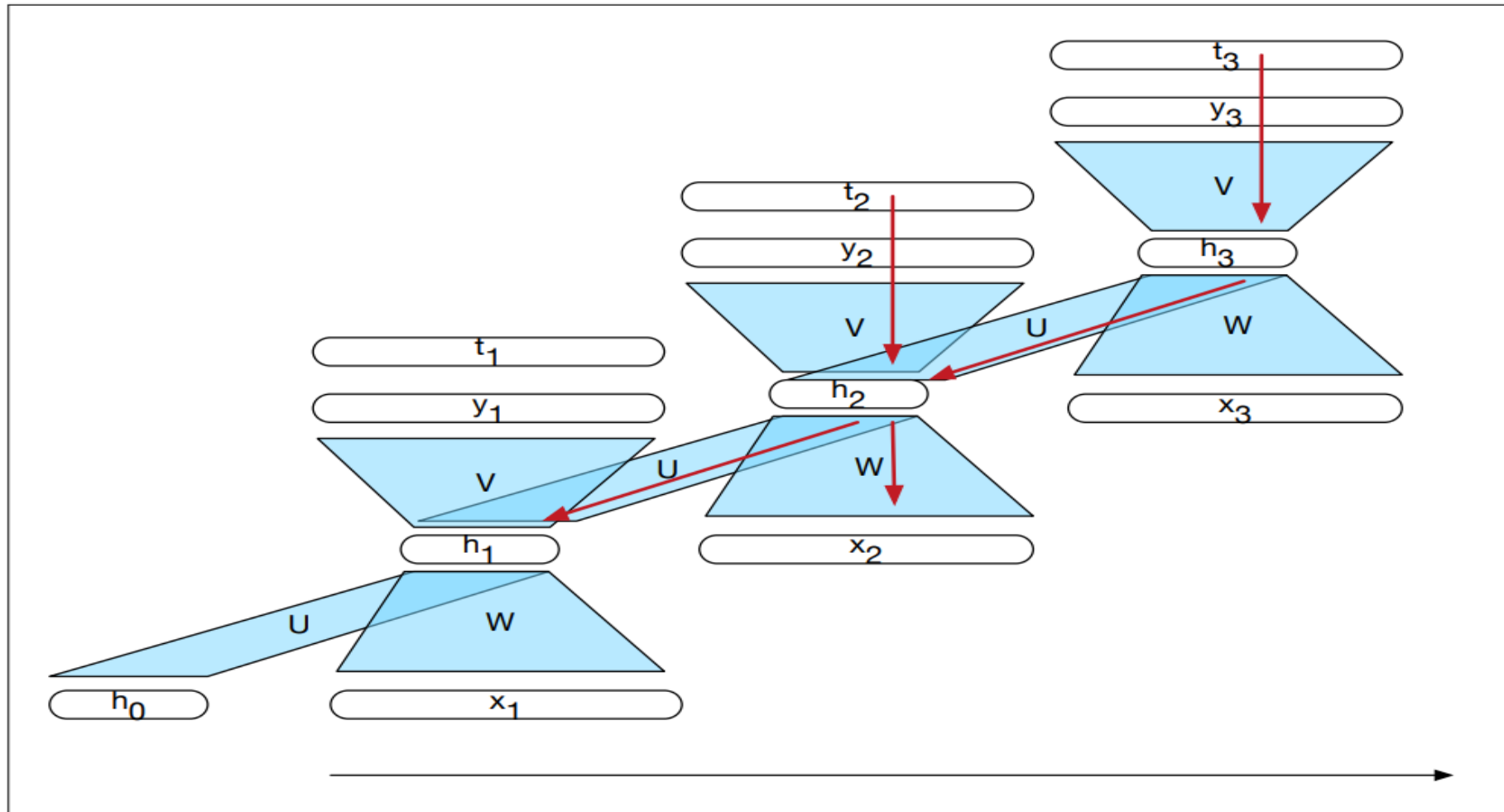


Figure 9.6 The backpropagation of errors in an SRN. The t_i vectors represent the targets for each element of the sequence from the training data. The red arrows illustrate the flow of backpropagated errors required to calculate the updates for U , V and W at time 2. The two incoming arrows converging on h_2 signal that these errors need to be summed.

Example

- [https://medium.com/@sachinsoni600517/recurrent-neural-networks-rnn-from-basic-to-advanced-1da22aafa009#:~:text=Many%20to%20Many%20%3A-,In%20recurrent%20neural%20networks%20\(RNNs\)%2C%20a%20%E2%80%9Cmany%2D,elements%20processed%20over%20time%20steps](https://medium.com/@sachinsoni600517/recurrent-neural-networks-rnn-from-basic-to-advanced-1da22aafa009#:~:text=Many%20to%20Many%20%3A-,In%20recurrent%20neural%20networks%20(RNNs)%2C%20a%20%E2%80%9Cmany%2D,elements%20processed%20over%20time%20steps)

.

Backpropagation

Weight updates

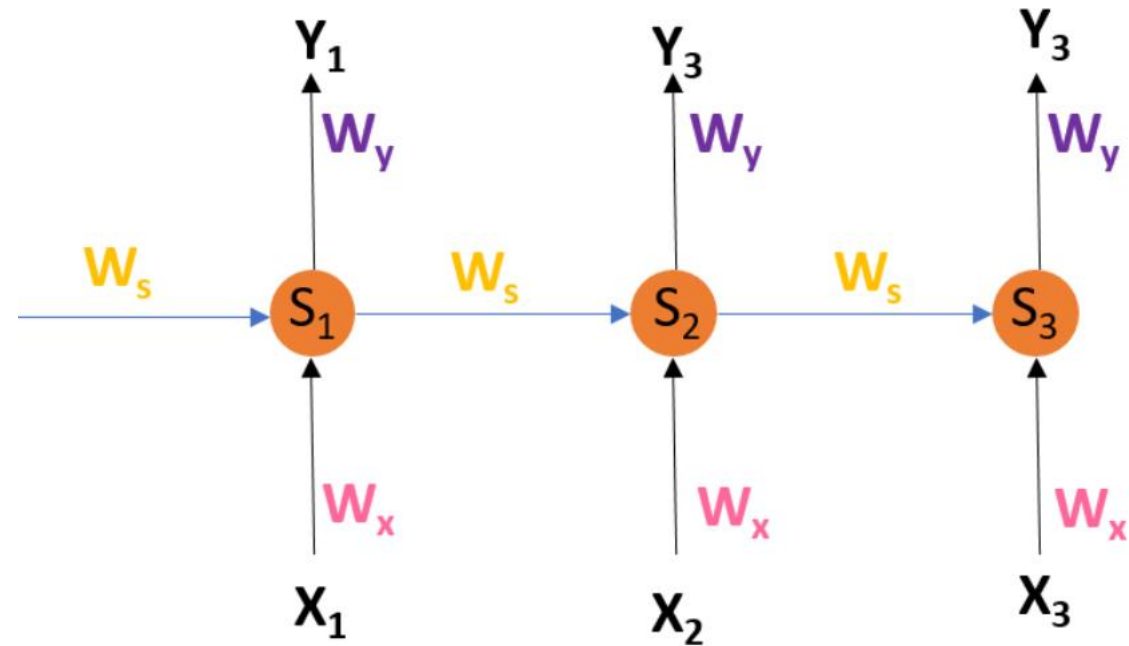
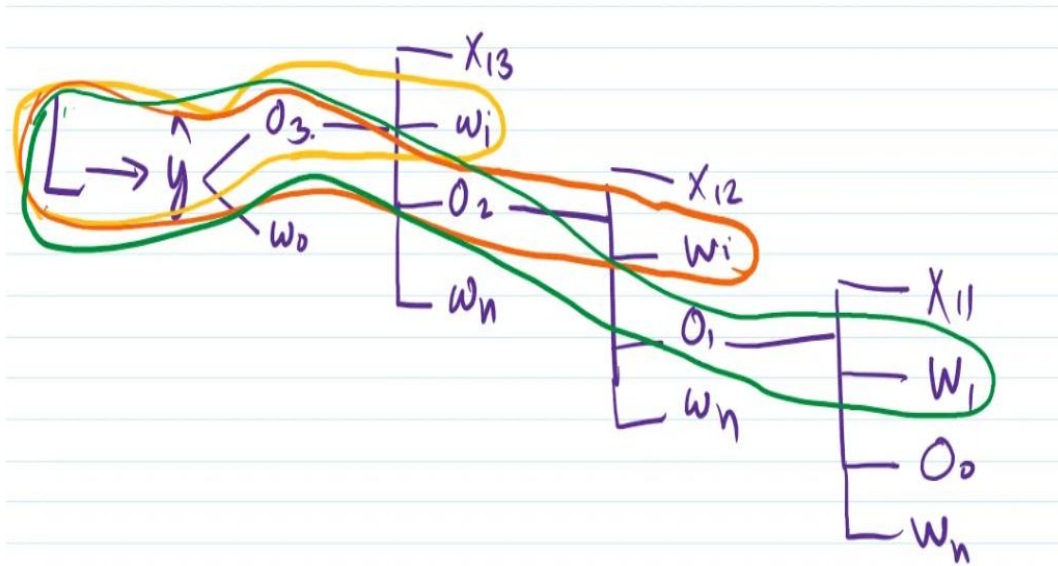
- Using the gradients, the weights in the actual RNN are adjusted in a way that will minimize the error in future predictions.

The image shows handwritten notes on a blackboard. At the top left, it says "L min" and "gradient descent" with "gradient" underlined. Below this, the weights w_i , w_h , and w_o are listed. An arrow points from the text "gradient descent" to the update equations. On the top right, there is a small note "L = -y". The update equations are:

$$\underline{w_i} = \underline{w_i} - \eta \left[\frac{\partial L}{\partial w_i} \right]$$
$$\underline{w_h} = \underline{w_h} - \eta \left[\frac{\partial L}{\partial w_h} \right]$$
$$\underline{w_o} = \underline{w_o} - \eta \left[\frac{\partial L}{\partial w_o} \right]$$

The gradients in the equations are enclosed in blue boxes.

Example (input sequence length=3)



RNN Architecture

Some notations:

$$h1 = O_1 = f(X_{11} \cdot W_i + O_0 \cdot W_h + b)$$

$$h2 = O_2 = f(X_{12} \cdot W_i + O_1 \cdot W_h + b)$$

$$h3 = O_3 = f(X_{13} \cdot W_i + O_2 \cdot W_h + b)$$

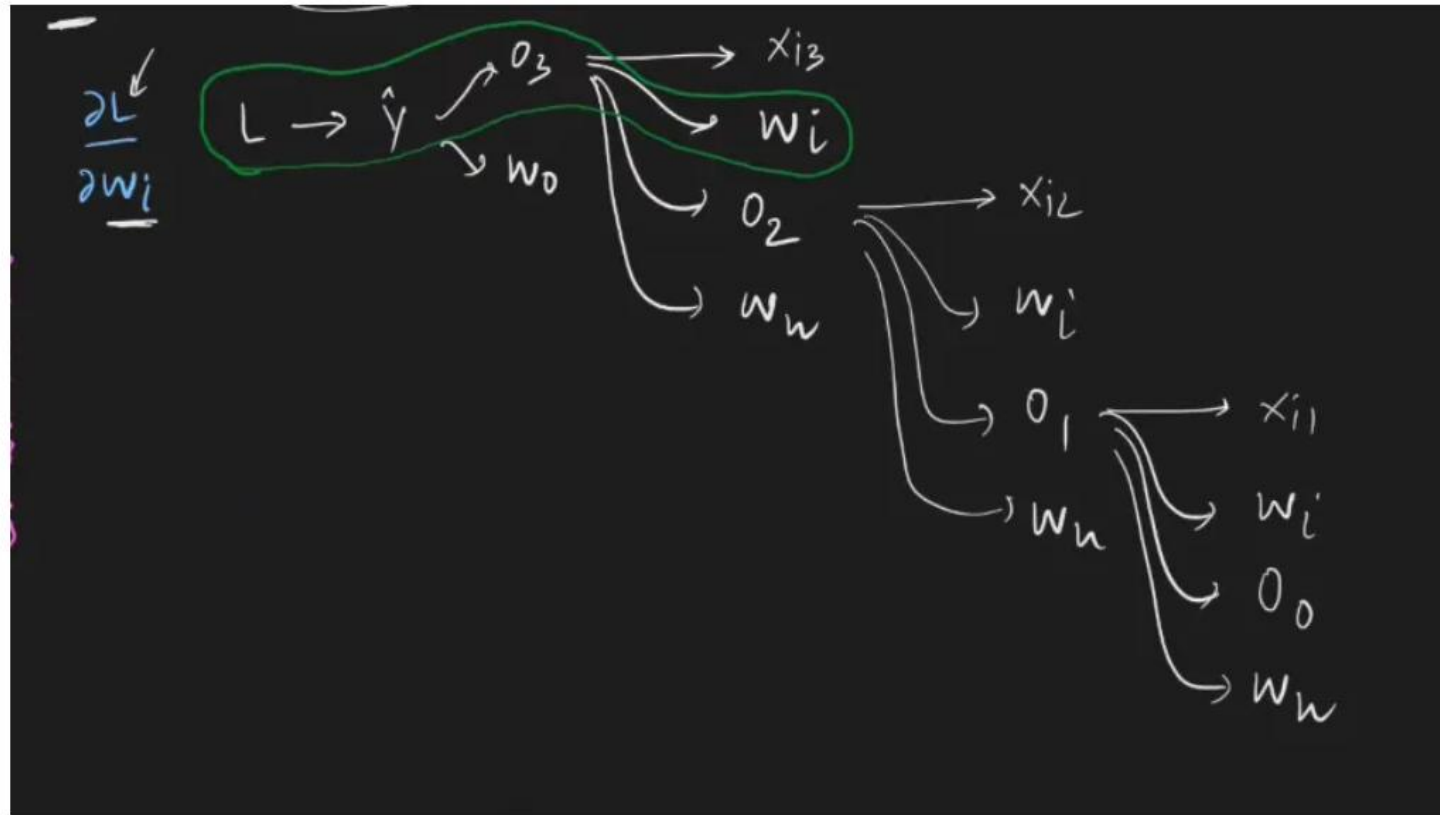
$$\hat{y} = f(O_3 \cdot W_o + b)$$

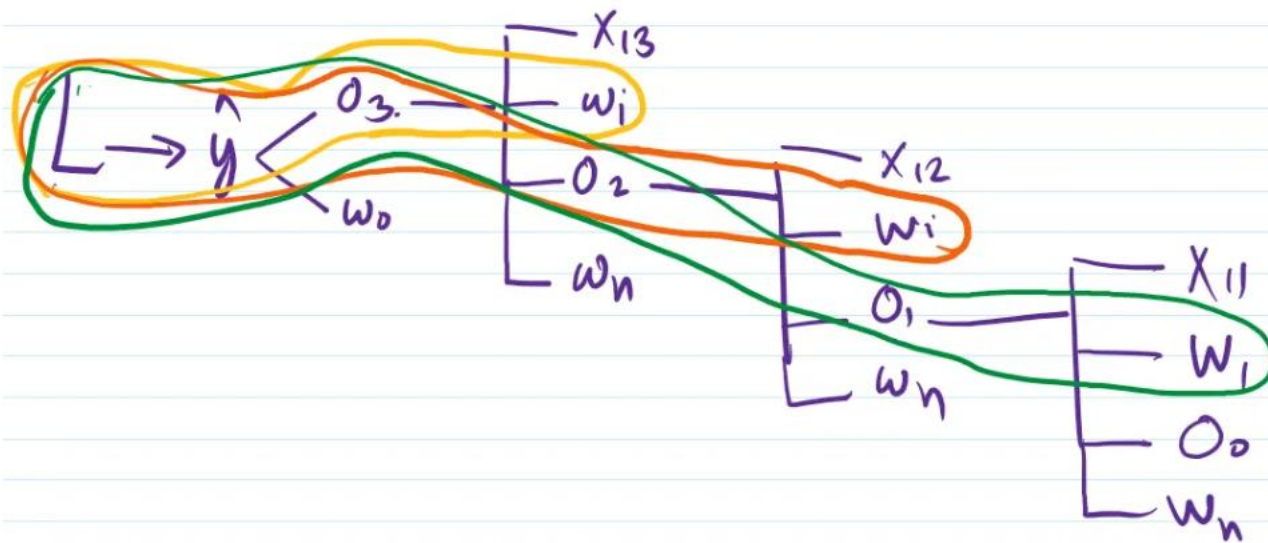
Derivative of Loss w.r.t output matrix W_o (v)


$$L \rightarrow \hat{y} \begin{cases} \rightarrow O_3 \\ \rightarrow W_o \end{cases} \Rightarrow \frac{\partial L}{\partial W_o} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_o}$$

This tells us that L is dependent on $\hat{y}$ and $\hat{y}$ is dependent on O_3 and W_o .

Dependency (W_i or W)





And after understanding above dependency we can write the gradient w.r.t. to w_i in the following way :

$$\boxed{\frac{\partial L}{\partial w_i}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial w_i} +$$

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial w_i} +$$

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial w_i}$$

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^3 \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_j} \frac{\partial o_j}{\partial w_i}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial w_1} \rightarrow \text{But } \frac{\partial \hat{y}}{\partial o_1} = \left(\frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \right)$$

$$\therefore \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_1}$$

Derivative of Loss w.r.t hidden-hidden matrix
 W_h or U

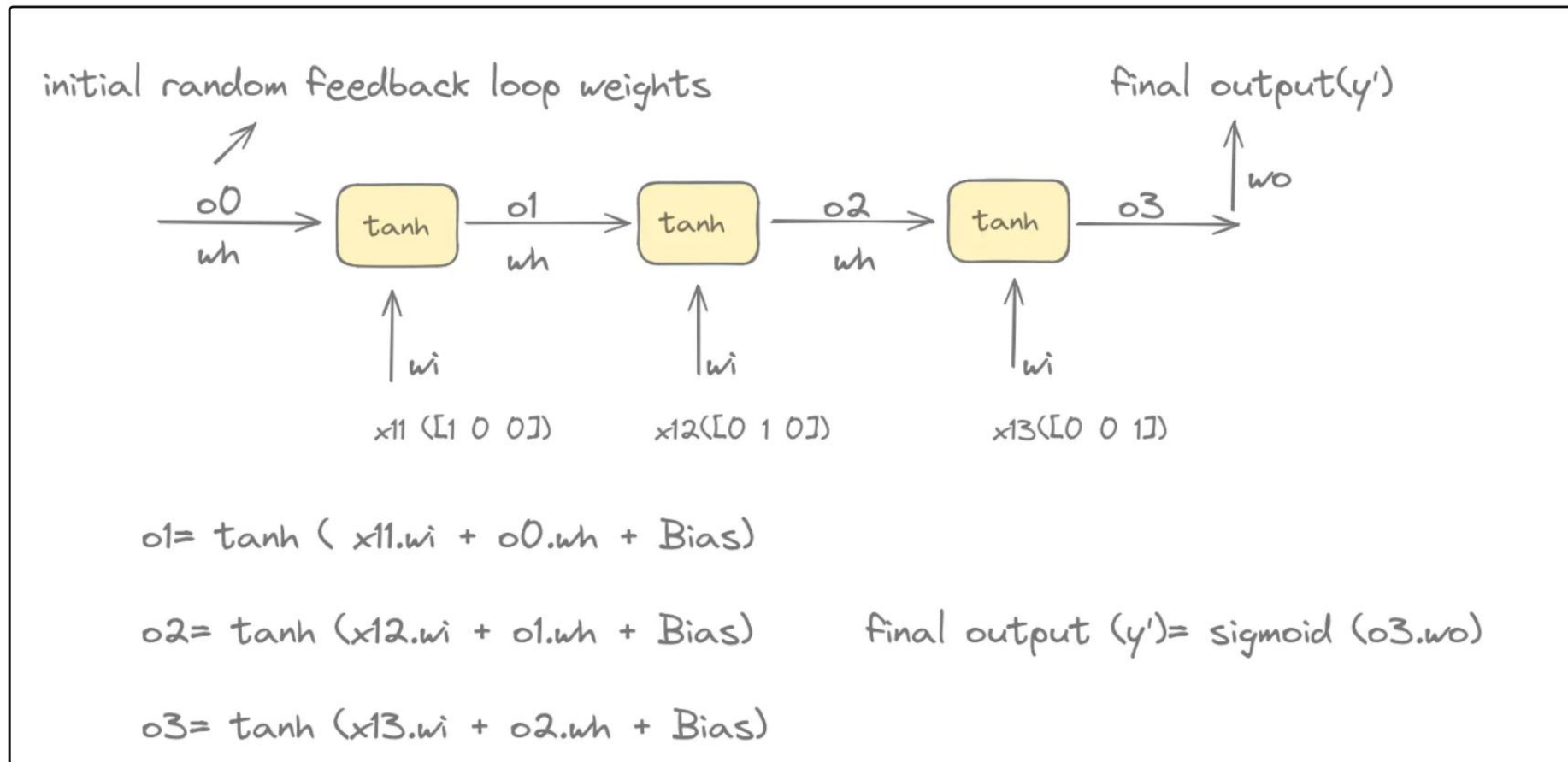
$$\frac{\partial L}{\partial W_h} = \sum_{j=1}^n \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_j} \cdot \frac{\partial o_j}{\partial W_h}$$

$n = \text{timestamp}$

Reference

- <https://medium.com/@abhishekjainindore24/backpropagation-in-rnn-abd87b940a7d>

Example (Many-to-one)



We can also write above equation in the following way also :

$$\frac{\partial \mathcal{L}}{\partial w_i} = \sum_{j=1}^n \frac{\partial \mathcal{L}}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial o_j} \frac{\partial o_j}{\partial w_i}$$

Error Calculation

- the difference (error) between the network's final output and the desired output.

And the loss function is calculated by using log loss

$$L = -y_i \log(y'_i) - (1-y_i)\log(1-y'_i)$$

where , y_i = Actual ground truth value

y'_i = Predicted value by model